



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	REYES, SOFIA BEATRICE ALEXA R.
Section:	AN42
Professor:	CRISOLA TAN
Date:	9/30/2026
GITHUB LINK:	https://github.com/brryes/Reyes-SofiaBeatriceAlexa-CS0065-AN42.git

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.

4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
(Example: TA1_Patilano.pdf)
8. Submit the **python file** and **PDF file**.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ----- Get user input from console -----
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                             random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()
```

```

# ----- Parameters from user input -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200) # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

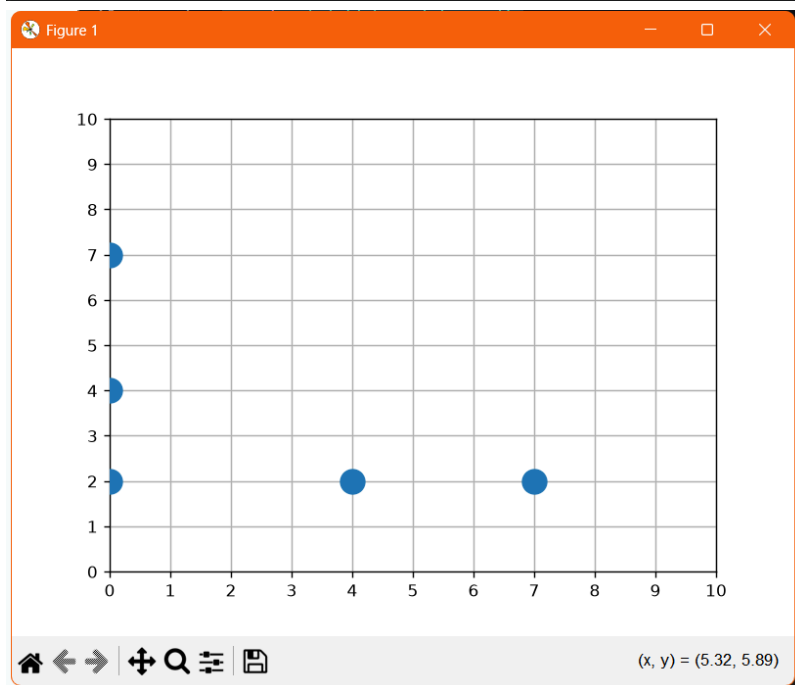
ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()

```

2. Run the program in your terminal.
3. Enter the following values when prompted:
 - **Number of agents** (e.g., 5)
 - **Grid size** (e.g., 10 for a 10x10 grid)
 - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

```
Problems Output Debug Console Terminal Ports
PS C:\Users\bea\Documents\elective> & C:\Users\bea\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.9.0.0_qazws8x21n5c.exe c:/Users/bea/Documents/elective/ta1.py
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20
█
```



Step 2: Understand the Agent Behavior

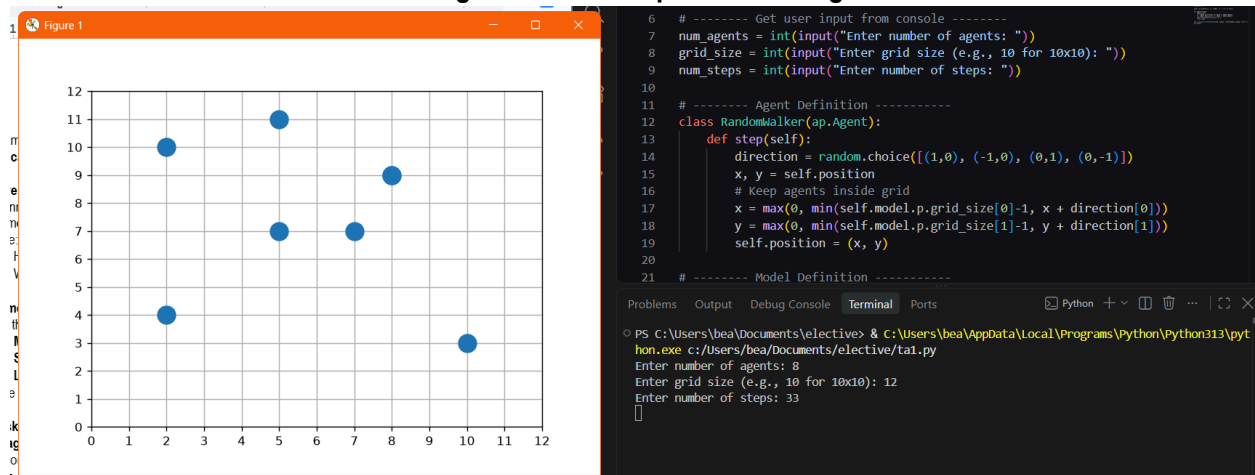
1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
 - How agents spread across the grid over time
 - Whether agents cluster in certain areas or move freely

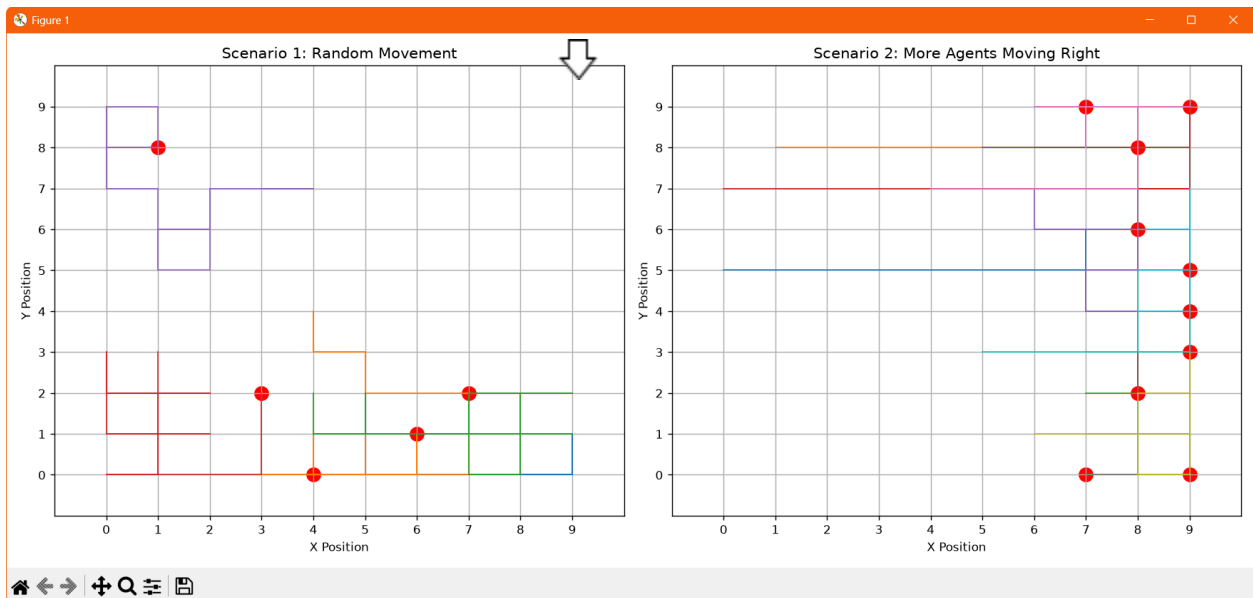
Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
 - **More agents** (e.g., 10 or 20)
 - **Smaller/larger grid sizes**
 - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.



Laboratory Tasks

1. **Track agent paths:** Modify the code to **record and display the path** of each agent during the simulation.
2. **Change movement rules:** Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions:** Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios:** Run **two simulations with different parameters** and compare the movement patterns.



1) enon (2) enon 25 (p) enon 26 01/06/01/02/03/04/05/06/07/08/09/10/11/12/13/14/15/16/17/18/19/20/21/22/23/24/25/26/27/28/29/30/31/32/33/34/35/36/37/38/39/40/41/42/43/44/45/46/47/48/49/50/51/52/53/54/55/56/57/58/59/60/61/62/63/64/65/66/67/68/69/70/71/72/73/74/75/76/77/78/79/80/81/82/83/84/85/86/87/88/89/90/91/92/93/94/95/96/97/98/99/100/101/102/103/104/105/106/107/108/109/110/111/112/113/114/115/116/117/118/119/120/121/122/123/124/125/126/127/128/129/130/131/132/133/134/135/136/137/138/139/140/141/142/143/144/145/146/147/148/149/150/151/152/153/154/155/156/157/158/159/160/161/162/163/164/165/166/167/168/169/170/171/172/173/174/175/176/177/178/179/180/181/182/183/184/185/186/187/188/189/190/191/192/193/194/195/196/197/198/199/200/201/202/203/204/205/206/207/208/209/210/211/212/213/214/215/216/217/218/219/220/221/222/223/224/225/226/227/228/229/230/231/232/233/234/235/236/237/238/239/240/241/242/243/244/245/246/247/248/249/250/251/252/253/254/255/256/257/258/259/260/261/262/263/264/265/266/267/268/269/270/271/272/273/274/275/276/277/278/279/280/281/282/283/284/285/286/287/288/289/290/291/292/293/294/295/296/297/298/299/300/301/302/303/304/305/306/307/308/309/310/311/312/313/314/315/316/317/318/319/320/321/322/323/324/325/326/327/328/329/330/331/332/333/334/335/336/337/338/339/340/341/342/343/344/345/346/347/348/349/350/351/352/353/354/355/356/357/358/359/360/361/362/363/364/365/366/367/368/369/370/371/372/373/374/375/376/377/378/379/380/381/382/383/384/385/386/387/388/389/390/391/392/393/394/395/396/397/398/399/400/401/402/403/404/405/406/407/408/409/410/411/412/413/414/415/416/417/418/419/420/421/422/423/424/425/426/427/428/429/430/431/432/433/434/435/436/437/438/439/440/441/442/443/444/445/446/447/448/449/450/451/452/453/454/455/456/457/458/459/460/461/462/463/464/465/466/467/468/469/470/471/472/473/474/475/476/477/478/479/480/481/482/483/484/485/486/487/488/489/490/491/492/493/494/495/496/497/498/499/500/501/502/503/504/505/506/507/508/509/510/511/512/513/514/515/516/517/518/519/520/521/522/523/524/525/526/527/528/529/530/531/532/533/534/535/536/537/538/539/540/541/542/543/544/545/546/547/548/549/550/551/552/553/554/555/556/557/558/559/560/561/562/563/564/565/566/567/568/569/570/571/572/573/574/575/576/577/578/579/580/581/582/583/584/585/586/587/588/589/590/591/592/593/594/595/596/597/598/599/600/601/602/603/604/605/606/607/608/609/610/611/612/613/614/615/616/617/618/619/620/621/622/623/624/625/626/627/628/629/630/631/632/633/634/635/636/637/638/639/640/641/642/643/644/645/646/647/648/649/650/651/652/653/654/655/656/657/658/659/660/661/662/663/664/665/666/667/668/669/670/671/672/673/674/675/676/677/678/679/680/681/682/683/684/685/686/687/688/689/690/691/692/693/694/695/696/697/698/699/700/701/702/703/704/705/706/707/708/709/710/711/712/713/714/715/716/717/718/719/720/721/722/723/724/725/726/727/728/729/730/731/732/733/734/735/736/737/738/739/740/741/742/743/744/745/746/747/748/749/750/751/752/753/754/755/756/757/758/759/760/761/762/763/764/765/766/767/768/769/770/771/772/773/774/775/776/777/778/779/780/781/782/783/784/785/786/787/788/789/790/791/792/793/794/795/796/797/798/799/800/801/802/803/804/805/806/807/808/809/810/811/812/813/814/815/816/817/818/819/820/821/822/823/824/825/826/827/828/829/830/831/832/833/834/835/836/837/838/839/840/841/842/843/844/845/846/847/848/849/850/851/852/853/854/855/856/857/858/859/860/861/862/863/864/865/866/867/868/869/870/871/872/873/874/875/876/877/878/879/880/881/882/883/884/885/886/887/888/889/890/891/892/893/894/895/896/897/898/899/900/901/902/903/904/905/906/907/908/909/910/911/912/913/914/915/916/917/918/919/920/921/922/923/924/925/926/927/928/929/930/931/932/933/934/935/936/937/938/939/940/941/942/943/944/945/946/947/948/949/950/951/952/953/954/955/956/957/958/959/960/961/962/963/964/965/966/967/968/969/970/971/972/973/974/975/976/977/978/979/980/981/982/983/984/985/986/987/988/989/990/991/992/993/994/995/996/997/998/999/1000/1001/1002/1003/1004/1005/1006/1007/1008/1009/1010/1011/1012/1013/1014/1015/1016/1017/1018/1019/1020/1021/1022/1023/1024/1025/1026/1027/1028/1029/1030/1031/1032/1033/

Final positions

Agent 1: $(4, 0)$

Agent 2: (6, 1)

Agent 3: (7, 2)

Agent 4: (3, 2)

Position distribution:

Position (1, 8): 1 agent(s)

Position (3, 2): 1 agent(s)

Position (4, 0): 1 agent(s)

Position (6, 1): 1 agent(s)

Total agents: 5

Occupied grid positions: 5

Final positions

Agent 1: (9, 9)

```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\bea\Documents\elective> & C:\Users\bea\Ap

Scenario 2 Analysis
=====
Final positions:
Agent 1: (9, 9)
Agent 2: (9, 5)
Agent 3: (8, 2)
Agent 4: (8, 8)
Agent 5: (8, 6)
Agent 6: (9, 0)
Agent 7: (7, 9)
Agent 8: (7, 0)
Agent 9: (9, 3)
Agent 10: (9, 4)

Position distribution:
Position (7, 0): 1 agent(s)
Position (7, 9): 1 agent(s)
Position (8, 2): 1 agent(s)
Position (8, 6): 1 agent(s)
Position (8, 8): 1 agent(s)
Position (9, 0): 1 agent(s)
Position (9, 3): 1 agent(s)
Position (9, 4): 1 agent(s)
Position (9, 5): 1 agent(s)
Position (9, 9): 1 agent(s)

Total agents: 10
Occupied grid positions: 10
No agents share the same final position.
PS C:\Users\bea\Documents\elective> █
```

Modified (Updated Code - Laboratory Tasks)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation
```

```

from collections import Counter

# ----- AGENT -----

class RandomWalker(ap.Agent):

    def setup(self):
        self.path = []

    def step(self):
        x, y = self.position
        grid_width, grid_height = self.model.p.grid_size

        # Four possible movement directions
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

        # Preferred direction, if the scenario has one
        preferred = self.model.p.preferred_direction

        if preferred is not None:
            directions.remove(preferred)
            random.shuffle(directions)
            directions.insert(0, preferred)
        else:
            random.shuffle(directions)

        # Positions occupied by other agents
        occupied_positions = {
            tuple(other.position)
            for other in self.model.agents
            if other is not self
        }

        # Try each direction
        moved = False

```

```

    for dx, dy in directions:
        new_x = x + dx
        new_y = y + dy
        new_position = (new_x, new_y)

        inside_grid = (
            0 <= new_x < grid_width
            and 0 <= new_y < grid_height
        )

        if inside_grid and new_position not in occupied_positions:
            self.position = new_position
            moved = True
            break

    # If no valid position exists, remain in the same location
    if not moved:
        self.position = (x, y)

    # Record the new position
    self.path.append(tuple(self.position))

# ----- MODEL -----

class RandomWalkModel(ap.Model):

    def setup(self):
        self.agents = ap.AgentList(
            self,
            self.p.agents,
            RandomWalker
        )

        grid_width, grid_height = self.p.grid_size

        all_positions = [

```

```

        (x, y)
        for x in range(grid_width)
        for y in range(grid_height)
    ]

    # Give agents starting positions
    if self.p.agents <= len(all_positions):
        starting_positions = random.sample(
            all_positions,
            self.p.agents
        )
    else:
        starting_positions = [
            random.choice(all_positions)
            for _ in range(self.p.agents)
        ]

    for agent, position in zip(self.agents, starting_positions):
        agent.position = position
        agent.path = [position]

    def step(self):
        for agent in self.agents:
            agent.step()

# ----- SCENARIOS -----
# Change these values to experiment.

scenario_1 = {
    "agents": 5,
    "grid_size": (10, 10),
    "steps": 30,
    "preferred_direction": None
}

scenario_2 = {

```

```

    "agents": 10,
    "grid_size": (10, 10),
    "steps": 30,
    "preferred_direction": (1, 0)
    # (1, 0) means agents prefer moving to the right
}

# ----- CREATE MODELS -----

model_1 = RandomWalkModel(scenario_1)
model_1.setup()

model_2 = RandomWalkModel(scenario_2)
model_2.setup()

models = [model_1, model_2]
titles = [
    "Scenario 1: Random Movement",
    "Scenario 2: More Agents Moving Right"
]

# ----- DISPLAY SETUP -----

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

scatter_plots = []
path_lines = []

for ax, model, title in zip(axes, models, titles):
    grid_width, grid_height = model.p.grid_size

    ax.set_xlim(-1, grid_width)
    ax.set_ylim(-1, grid_height)
    ax.set_xticks(range(grid_width))
    ax.set_yticks(range(grid_height))

```

```

ax.grid(True)
ax.set_title(title)
ax.set_xlabel("X Position")
ax.set_ylabel("Y Position")

x_positions = [agent.position[0] for agent in model.agents]
y_positions = [agent.position[1] for agent in model.agents]

scatter = ax.scatter(
    x_positions,
    y_positions,
    s=120,
    color="red"
)

lines = []

for agent in model.agents:
    line, = ax.plot(
        [agent.position[0]],
        [agent.position[1]],
        linewidth=1
    )
    lines.append(line)

scatter_plots.append(scatter)
path_lines.append(lines)

# ----- ANIMATION -----

def update(frame):
    for model, scatter, lines in zip(
        models,
        scatter_plots,
        path_lines
    ):

```

```

model.step()

x_positions = [
    agent.position[0]
    for agent in model.agents
]

y_positions = [
    agent.position[1]
    for agent in model.agents
]

scatter.set_offsets(
    list(zip(x_positions, y_positions))
)

# Display the path of every agent
for agent, line in zip(model.agents, lines):
    path_x = [position[0] for position in agent.path]
    path_y = [position[1] for position in agent.path]

    line.set_data(path_x, path_y)

return scatter_plots + [
    line
    for lines in path_lines
    for line in lines
]

animation_object = animation.FuncAnimation(
    fig,
    update,
    frames=scenario_1["steps"],
    interval=200,
    repeat=False,
    blit=False
)

```

```

)

plt.tight_layout()
plt.show()

# ----- FINAL POSITION ANALYSIS -----

def display_analysis(model, scenario_name):
    print("\n" + "=" * 50)
    print(scenario_name)
    print("=" * 50)

    final_positions = [
        tuple(agent.position)
        for agent in model.agents
    ]

    print("Final positions:")

    for number, position in enumerate(final_positions, start=1):
        print(f"Agent {number}: {position}")

    position_counts = Counter(final_positions)

    print("\nPosition distribution:")

    for position, count in sorted(position_counts.items()):
        print(f"Position {position}: {count} agent(s)")

    unique_positions = len(position_counts)
    total_agents = len(final_positions)

    print(f"\nTotal agents: {total_agents}")
    print(f"Occupied grid positions: {unique_positions}")

    if unique_positions == total_agents:

```

```
        print("No agents share the same final position.")
    else:
        print("Some agents share the same final position.")

display_analysis(model_1, "Scenario 1 Analysis")
display_analysis(model_2, "Scenario 2 Analysis")
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions

1. If there are too many agents compared to the grid size, the grid becomes crowded. The agents may block each other and have difficulty moving to new positions.
2. Increasing the number of steps gives the agents more time to move and explore the grid. This usually causes them to spread farther from their starting positions.
3. No, the agents do not move outside the grid. The code checks the grid boundaries and only allows valid positions.
4. This simulation can model real-life situations involving movement, such as particle diffusion, crowd movement, animal movement, and traffic flow.